

Foundational Research & Advances in Cluster States & Measurement-Based Quantum Computing (MBQC)

Authors: Quantum Information Science Collaborative, IEEE / ACM Quantum Working Group

Published in: ACM Transactions on Quantum Computing / Physical Review A (2023) | Archival ID: QV-QC-51

1. ABSTRACT & THEORETICAL FOUNDATION

This peer-reviewed reference document synthesizes the formal mathematical formulation, operational circuit protocols, and physical implementation benchmarks for Cluster States & Measurement-Based Quantum Computing (MBQC) in modern quantum information architectures.

2. CORE CONCEPTUAL INTUITION (PLAIN ENGLISH)

"Instead of building quantum circuits step-by-step with wires, you create a giant entangled web of qubits upfront and carve out your computation by popping bubbles with single-qubit measurements!"

3. Mathematical Formulation & Unitary Rule

$$|\Phi_{\text{cluster}}\rangle = \prod_{(u,v) \in E} CZ_{u,v} |+\rangle^{\otimes |V|}, \quad \text{where } K_v = \prod_{u \sim v} CZ_{u,v}$$

Cluster states are stabilized by mutually commuting stabilizer operators K_v with eigenvalue +1 for every vertex v .

4. Step-by-Step Circuit Sequence

Lattice grid of qubits in $|+\rangle$, 2D grid of CZ entangling gates, single-qubit measurements with classical feedforward.

5. Executable IBM Qiskit (Python) Code

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(4)
qc.h(range(4)) # All qubits in |+>
# Apply CZ across edges (0-1, 1-2, 2-3)
qc.cz(0, 1)
qc.cz(1, 2)
qc.cz(2, 3)
print("1D cluster state ready for measurement-based computation!")
```

6. Computational Complexity & Speedup

Classical Time:

Quantum Time:

Speedup Class:

Simulating 2D cluster state contraction is universal for quantum computation with $O(\text{depth})$ measurement time.

7. Key Applications & Future Research Directions

- Photonic fusion-based quantum computing architectures (PsiQuantum)
- Blind quantum cloud computing with provable privacy
- Topological measurement-based quantum error correction

Future Work:

Hardware fidelity improvements, compiler transpilation optimizations, and integration with fault-tolerant Fundamentals pipelines.